VOLUME 11 OF 12

IT STUDY HUB

COMPLETE JAVA PROGRAMMING SERIES

MODULE 11

SPRING BOOT & ENTERPRISE JAVA

Authored by:

Mithila Rani Saha & Dithsa Dutta

www.itstudyhub.dpdns.org

Welcome to Module 11

1. About This Module

Welcome to Volume 11 of the IT Study Hub Complete Java Programming Series. In Module 10, you built a simple Backend Application using pure JDBC. While highly educational, writing raw SQL strings and manually handling HTTP servers is exhausting and dangerous in real-world enterprise development.

Enter **Spring Boot**. Spring is the most powerful and popular framework for building enterprise-grade Java applications. It eliminates massive amounts of boilerplate code, automatically configures databases, handles HTTP routing, and secures your application. This module shifts your mindset from "writing every line of code" to "orchestrating an intelligent framework."

2. Learning Outcomes

- Understand Frameworks, Inversion of Control (IoC), and Dependency Injection (DI).
- Distinguish between the Spring Framework and Spring Boot.
- Build RESTful Web Services using `@RestController`, handling GET, POST, PUT, and DELETE methods.
- Extract data from HTTP requests using `@PathVariable`, `@RequestParam`, and `@RequestBody`.
- Implement the Enterprise 3-Tier Architecture (Controller → Service → Repository).
- Automate Database interactions without SQL strings using **Spring Data JPA** and **Hibernate**.
- Sanitize data using Data Transfer Objects (DTOs) and Validation.
- Handle global exceptions elegantly with `@ControllerAdvice`.
- Grasp the fundamentals of Authentication via Spring Security.

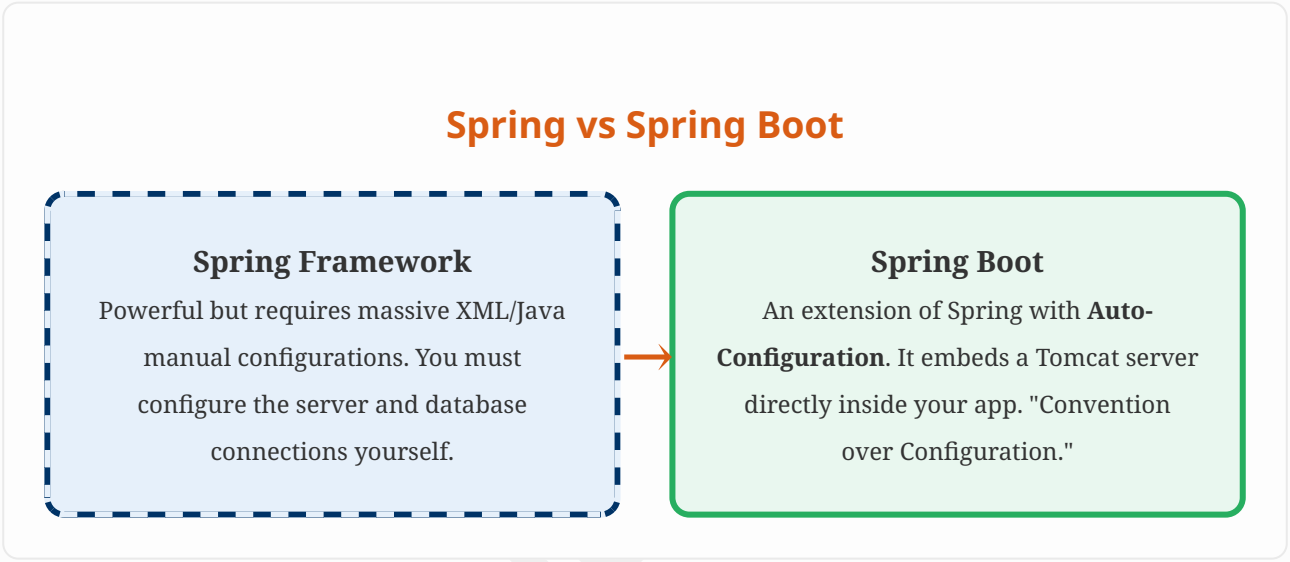
3. Module Coverage

We begin by demystifying "Dependency Injection" and "Inversion of Control". Next, we map out the strict 3-Tier architecture. We dive into code, creating REST APIs, extracting variables, and validating JSON bodies. We transition to the database layer, replacing our old JDBC code with magical Spring Data JPA interfaces. Finally, we synthesize everything by building a "Complete RESTful Backend Using Spring Boot", followed by comprehensive interview preparation.

IT STUDY HUB

4. Main Notes: Introduction to Spring Framework

A **Framework** is a massive, pre-written body of code that dictates the architecture of your application. Instead of you writing the main loop and calling library functions, the framework owns the main loop and calls *your* code when necessary.



Spring vs Spring Boot Comparison

Feature	Spring Framework	Spring Boot
Configuration	Requires heavy XML or Java manual config.	Auto-configuration. It guesses what you need.
Server	Requires you to install external Tomcat/server.	Embeds Tomcat directly. Just click "Run".
Dependencies	Manual version matching in Maven.	Uses "Starters" (e.g., `spring-boot-starter-web`) to group compatible libraries.

5. Inversion of Control & Dependency Injection

IT STUDY HUB

Concept: Dependency Injection (DI) & IoC

A design pattern where objects are handed their dependencies (tools they need) rather than creating them internally.

> ***Real-Life Analogy:***
> *Imagine a Chef (a Class) who needs a Knife (a Dependency) to cook. Without DI, the Chef has to forge the knife from iron before cooking (using the `new` keyword). With DI, the Restaurant Manager (IoC Container) hands a pre-made Knife to the Chef the moment they walk into the kitchen. The control is 'Inverted'.*

> **Simple Explanation:**
> Inversion of Control (IoC) means Spring takes control of creating objects (called 'Beans'). Dependency Injection (DI) is how Spring hands those objects to the classes that need them, typically via the class constructor.

Syntax / Structure:

```
@Component
public class Knife { ... }

@Service
public class Chef {
    private final Knife knife;

    // Spring automatically injects the Knife Bean here!
    public Chef(Knife knife) {
        this.knife = knife;
    }
}
```

Example Code:

```
import org.springframework.stereotype.Service;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.RestController;

// 1. We mark this class so Spring manages it (creates a Bean)
@Service
class EmailService {
```

```
    public void sendEmail() {
        System.out.println("Email sent successfully!");
    }
}

// 2. We inject the Service into our Controller
@RestController
class NotificationController {

    private final EmailService emailService;

    // 3. Constructor Injection (Best Practice!)
    @Autowired
    public NotificationController(EmailService emailService) {
        this.emailService = emailService;
    }

    public void triggerNotification() {
        emailService.sendEmail();
    }
}
```

```
Expected Output / API Response:
Email sent successfully!
```

Code Explanation:
We never typed `new EmailService()`. The `@Service` annotation told Spring to create an EmailService object and hold it in its internal 'IoC Container'. When Spring created `NotificationController`, it saw the constructor required an `EmailService`, so it automatically 'Injected' it.

> ***Real-world Use:***
> *Injecting Database Repositories into Business Logic Services. If the database type changes, you only change it in one place, and Spring injects the new one everywhere automatically.*

Common Mistakes:

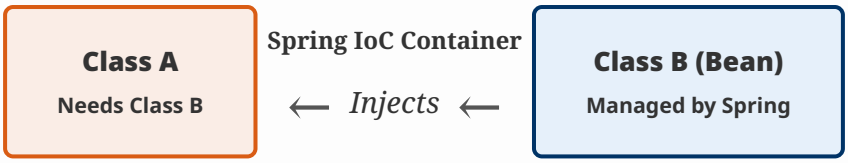
- Using Field Injection (putting `@Autowired` directly on the variable instead of the constructor). This makes testing difficult and hides dependency requirements.

Best Practices:

- Always use Constructor Injection. Mark your injected fields as `private final`. As of newer Spring versions, `@Autowired` is optional on single constructors, but good for clarity.

Practice Task:
Create a `@Service` called `PaymentService`. Inject it via the constructor into a `CheckoutController`.

Dependency Injection Flow

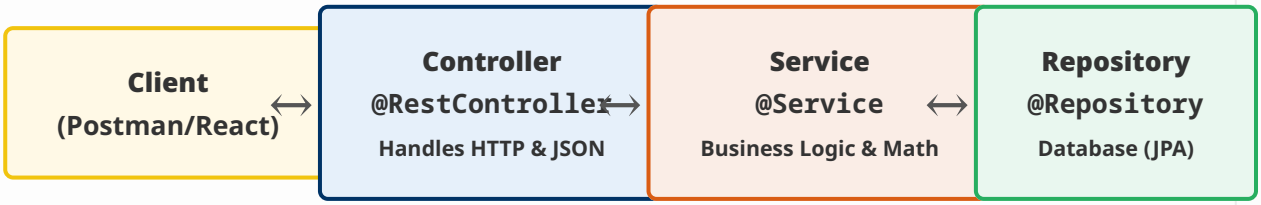


---

6. Spring Boot Enterprise Architecture

Professional applications are divided into strictly separated layers. This **3-Tier Architecture** ensures that if you change your database, you don't have to rewrite your HTTP web code.

Controller → Service → Repository Workflow



Layer Comparison

Layer	Annotation	Responsibility (Real-Life Analogy)
Controller	`@RestController`	**The Waiter:** Takes the customer's order (HTTP Request), hands it to the kitchen, and returns the food (JSON Response).
Service	`@Service`	**The Chef:** Executes business rules. Calculates taxes, checks if items are in stock, processes the actual logic.
Repository	`@Repository`	**The Pantry:** Only talks to the database. Fetches raw ingredients (Data) and saves new data. No math or logic!

7. Building REST APIs (Controllers)

IT STUDY HUB

Concept: REST Controllers and HTTP Methods

Exposing your Java code to the internet so web browsers and mobile apps can trigger your methods.

> ***Real-Life Analogy:***
> *A Controller is like a telephone switchboard. When a call (HTTP Request) comes in on a specific extension number (URL Path), the switchboard routes the call to the correct desk (Java Method).*

> **Simple Explanation:**
> We annotate a class with `@RestController`. Then we map HTTP verbs (GET, POST) to specific URL paths using `@GetMapping("/path")`. Spring automatically converts the Java object we return into JSON text for the client.

Syntax / Structure:

```
@RestController
@RequestMapping("/api/users")
public class UserController {
    @GetMapping
    public List getUsers() { ... }
}
```

Example Code:

```
import org.springframework.web.bind.annotation.*;
import java.util.Arrays;
import java.util.List;

@RestController
@RequestMapping("/api/greetings") // Base URL for this entire class
public class GreetingController {

    // 1. Handles HTTP GET requests to /api/greetings/hello
    @GetMapping("/hello")
    public String sayHello() {
        return "Hello, Welcome to Spring Boot!";
    }
```

```
    // 2. Returns a List. Spring automatically converts it to a JSON Array!
    @GetMapping("/list")
    public List getNames() {
        return Arrays.asList("Alice", "Bob", "Charlie");
    }
}
```

```
Expected Output / API Response:
GET /api/greetings/list

[
  "Alice",
  "Bob",
  "Charlie"
]
```

Code Explanation:
`@RestController` tells Spring 'This class listens for web requests'. `@RequestMapping` sets a base URL. When a client performs a GET request to `/api/greetings/list`, Spring routes it to `getNames()`. Notice how we returned a Java `List`, but the output is valid JSON. Spring uses a library called 'Jackson' to do this translation automatically.

Real-world Use:
Providing data to a React.js frontend. The frontend calls your `@GetMapping` URL, receives the JSON, and displays it on the screen.

Common Mistakes:

- Forgetting `@RestController`. If you just use `@Controller`, Spring expects you to return an HTML webpage file instead of raw JSON data.

Best Practices:

- Keep URLs lowercase, plural, and use hyphens (e.g., `/api/student-records`). Never use verbs in the URL (e.g., `/api/getStudents` is bad practice; use `GET /api/students`).

Practice Task:
Create a `@RestController` mapped to `/api/math`. Create a `@GetMapping("/pi")` that returns the double 3.14159.

IT STUDY HUB

Concept: Extracting Data: Path Variables & Request Parameters

Reading variables sent by the client inside the URL.

> ***Real-Life Analogy:***
> *If a URL is an address on an envelope, a PathVariable is the Apartment Number directly in the address. A RequestParameter is an optional sticky note attached to the envelope saying '?color=blue'.*

> **Simple Explanation:**
> `@PathVariable` extracts values directly embedded in the URL path (e.g., `/users/5`). `@RequestParam` extracts values from the query string at the end of the URL (e.g., `/users?role=admin`).

Syntax / Structure:

```
@GetMapping("/users/{id}")
public User getUser(@PathVariable int id) { ... }

@GetMapping("/search")
public List search(@RequestParam String name) { ... }
```

Example Code:

```
import org.springframework.web.bind.annotation.*;

@RestController
@RequestMapping("/api/products")
public class ProductController {

    // Example 1: Path Variable (Required part of the URL)
    // Client calls: GET /api/products/101
    @GetMapping("/{id}")
    public String getProductById(@PathVariable int id) {
        return "Fetching product exactly matching ID: " + id;
    }

    // Example 2: Request Parameter (Optional query string)
```

```
    // Client calls: GET /api/products/search?category=Electronics&maxPrice=500
    @GetMapping("/search")
    public String searchProducts(
            @RequestParam String category,
            @RequestParam(defaultValue = "1000") int maxPrice) {

        return "Searching for " + category + " under $" + maxPrice;
    }
}
```

```
Expected Output / API Response:
GET /api/products/search?category=Shoes

Searching for Shoes under $1000
```

Code Explanation:
In Example 1, the `{id}` in the mapping binds directly to the `@PathVariable int id`. In Example 2, we requested a `category` and `maxPrice`. Because the client didn't provide `maxPrice` in the URL, Spring utilized the `defaultValue` of 1000.

> ***Real-world Use:***
> *`@PathVariable` is used to load a specific user's profile profile page. `@RequestParam` is used for search bars and filtering results.*

Common Mistakes:

- Putting the `?` in the `@GetMapping` annotation. Never do this! `@GetMapping("/search?name=...")` is invalid. Spring handles the `?` automatically via `@RequestParam`.

Best Practices:

- Use `@PathVariable` to identify a specific resource. Use `@RequestParam` to filter, sort, or paginate a list of resources.

Concept: Receiving Data: @RequestBody (POST & PUT)

Accepting complex JSON objects sent by the client to save to the database.

> ***Real-Life Analogy:***
> *When you submit a job application form online, all the text fields are bundled into a digital briefcase (JSON Body) and handed to the server. `@RequestBody` unpacks the briefcase into a Java Object.*

> **Simple Explanation:**
> For `POST` (creating) and `PUT` (updating), data is too large to fit in a URL. It is sent in the Request Body as JSON. Spring intercepts this JSON and automatically converts it into a Java Object using the `@RequestBody` annotation.

Syntax / Structure:

```
@PostMapping("/create")
public String createUser(@RequestBody User user) { ... }
```

Example Code:

```
import org.springframework.web.bind.annotation.*;

// Simple POJO (Plain Old Java Object)
class Product {
    public String name;
    public double price;
}

@RestController
@RequestMapping("/api/products")
public class InventoryController {

    // Client sends a POST request with JSON in the body
    @PostMapping
    public String addProduct(@RequestBody Product newProduct) {
        // Spring has already converted the JSON into the Product object!
        System.out.println("Saving " + newProduct.name + " to database...");
        return "Successfully created product: " + newProduct.name;
```

```
    }
}
```

```
Expected Output / API Response:
CLIENT SENDS JSON:
{
  "name": "Laptop",
  "price": 1200.0
}

API RESPONSE:
Successfully created product: Laptop
```

Code Explanation:
The Client sends a POST request. The HTTP Request Body contains a JSON object. Spring sees `@RequestBody Product`, looks at the JSON keys (`name`, `price`), matches them to the Java class fields, and instantiates the `Product` object for us automatically. This is called Deserialization.

> ***Real-world Use:***
> *Submitting a registration form, adding an item to a cart, or uploading a document.*

> **Common Mistakes:**
>
> - Forgetting the `@RequestBody` annotation. If you forget it, Spring will try to find the data in the URL, fail, and your Java object will be completely `null`!

> **Best Practices:**
>
> - Never accept a direct Database Entity object in the `@RequestBody`. Always use a Data Transfer Object (DTO) for security (explained later).

Practice Task:
Create a `Book` class (title, author). Create a `@PostMapping` endpoint that accepts a Book via `@RequestBody` and returns a confirmation string.

---

8. Database Layer: Spring Data JPA & Hibernate

In Module 10, writing JDBC SQL queries was tedious. **Spring Data JPA** (Java Persistence API) revolutionizes this. It is an abstraction layer on top of **Hibernate** (an ORM - Object Relational Mapper). It automatically writes and executes SQL queries for you based entirely on your Java classes!

JPA vs Hibernate

JPA (Java Persistence API)	Hibernate
The **Specification**. A set of rules and interfaces defined by Java.	The **Implementation**. The actual code that does the work of connecting to the database and generating SQL.
Analogy: The rulebook for how to play chess.	Analogy: The physical chessboard and pieces.

IT STUDY HUB

Concept: JPA Entities and Repositories

Mapping a Java Class to a Database Table, and creating a Repository to handle CRUD operations without writing SQL.

> ***Real-Life Analogy:***
> *An `@Entity` is a rubber stamp. You design it once in Java, and Hibernate stamps out a perfectly matching Table in MySQL. The `Repository` is the robotic filing clerk that saves and retrieves the stamped papers automatically.*

> **Simple Explanation:**
> We mark a class with `@Entity` to tell Spring to create a database table for it. We mark the primary key with `@Id`. Then, we create an interface extending `JpaRepository`. Spring magically generates the implementation of `save()`, `findAll()`, `findById()`, and `delete()` at runtime!

Syntax / Structure:

```
@Entity
public class User {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
}

public interface UserRepository extends JpaRepository { }
```

Example Code:

```
import jakarta.persistence.*;
import org.springframework.data.jpa.repository.JpaRepository;

// 1. THE ENTITY (Creates table 'employee')
@Entity
public class Employee {

    @Id // Primary Key
    @GeneratedValue(strategy = GenerationType.IDENTITY) // AUTO_INCREMENT
    private Long id;

    private String name;
```

```
    private double salary;

    // Getters, Setters, Constructors omitted for brevity...
}

// 2. THE REPOSITORY (Handles all SQL automatically!)
//  ->
public interface EmployeeRepository extends JpaRepository {

    // MAGIC: We just name the method correctly, and Spring writes the SQL!
    // Equivalent to: SELECT * FROM employee WHERE name = ?
    List findByName(String name);
}
```

```
Expected Output / API Response:
[No console output. Hibernate automatically creates the 'employee' table in MySQL behind
```

Code Explanation:
`@Entity` commands Hibernate to manage this class. `@Id` and `@GeneratedValue` configure the primary key. `EmployeeRepository` is just an interface! We don't write any code inside it. By extending `JpaRepository`, it instantly gains methods like `.save()`, `.deleteById()`, and `.findAll()`. We can even invent methods like `findByName()` and Spring generates the SQL via intelligent parsing.

> ***Real-world Use:***
> *Replacing hundreds of lines of brittle JDBC `PreparedStatement` and `ResultSet` boilerplate with a 2-line interface.*

> **Common Mistakes:**
>
> - Forgetting the empty Default Constructor in the Entity class. Hibernate absolutely requires an empty constructor to instantiate objects when fetching data from the database.

> **Best Practices:**
>
> - Keep Entity classes strictly for database mapping. Do not put business logic or complex math inside an Entity.

---

9. DTOs, Validation, and Exception Handling

Never expose your database Entities directly to the web client. If an `User` Entity has a password field, returning it directly in a Controller will send the password as JSON to the web browser! We solve this using **DTOs (Data Transfer Objects)**.

IT STUDY HUB

Concept: DTOs and Input Validation

Creating separate objects strictly for carrying data over the internet, and validating them before they reach the Service layer.

> ***Real-Life Analogy:***
> *When a politician travels, they don't carry their personal diary (Entity). They carry a carefully curated press release (DTO). Security checks the press release for bad words (Validation) before letting it go to the press.*

> **Simple Explanation:**
> We create a `StudentDTO` class that only contains safe fields (like name, excluding sensitive DB IDs). We use annotations from `jakarta.validation` like `@NotBlank` or `@Min` to set rules. In the controller, we add `@Valid` next to the `@RequestBody` to trigger the checks.

Syntax / Structure:

```
public class UserDTO {
    @NotBlank(message = "Name is required")
    private String name;
}

public String addUser(@Valid @RequestBody UserDTO dto) { ... }
```

Example Code:

```
import jakarta.validation.constraints.*;
import org.springframework.web.bind.annotation.*;

// 1. The DTO
class StudentDTO {
    @NotBlank(message = "Name cannot be empty!")
    public String name;

    @Min(value = 18, message = "Student must be at least 18 years old.")
    public int age;
}

@RestController
```

```
@RequestMapping("/api/students")
public class StudentController {

    // 2. Controller using @Valid
    @PostMapping
    public String registerStudent(@Valid @RequestBody StudentDTO request) {
        // If validation fails, Spring blocks execution and returns a 400 Bad Request
        return "Student " + request.name + " safely validated and registered!";
    }
}
```

```
Expected Output / API Response:
CLIENT SENDS JSON:
{
  "name": "",
  "age": 15
}

API RESPONSE (HTTP 400 Bad Request):
Validation failed: Student must be at least 18 years old.
```

Code Explanation:
The `@Valid` annotation tells Spring to check the `StudentDTO` against its validation rules BEFORE running the method. Because the name was blank and age < 18, Spring immediately rejected the HTTP request and threw a `MethodArgumentNotValidException`.

Real-world Use:
Validating email formats (`@Email`), ensuring passwords meet complexity requirements, preventing blank form submissions.

Common Mistakes:

- Putting validation annotations on the DTO, but forgetting the `@Valid` annotation in the Controller. Without `@Valid`, the checks are completely ignored!

Best Practices:

- Always validate incoming data at the Controller layer. The Service layer should assume data handed to it is already sanitized.

IT STUDY HUB

Concept: Global Exception Handling (@ControllerAdvice)

A single class that catches exceptions thrown anywhere in your application and formats a beautiful JSON error response for the client.

> ***Real-Life Analogy:***
> *A company's PR department. If any employee anywhere in the building makes a massive mistake (an Exception), the PR department intercepts it and issues a calm, formatted apology to the public, preventing the raw chaos from leaking out.*

> **Simple Explanation:**
> Instead of writing `try-catch` blocks in every single controller method, we create a class with `@ControllerAdvice`. If an exception occurs, Spring redirects it here. We use `@ExceptionHandler` to format the error into a clean JSON response instead of an ugly Java Stack Trace.

Syntax / Structure:

```
@ControllerAdvice
public class GlobalExceptionHandler {
    @ExceptionHandler(CustomException.class)
    public ResponseEntity handle(CustomException ex) { ... }
}
```

Example Code:

```
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

// Custom Runtime Exception
class ResourceNotFoundException extends RuntimeException {
    public ResourceNotFoundException(String msg) { super(msg); }
}

// Global Exception Handler
@ControllerAdvice
public class GlobalExceptionHandler {

    // Catch specific exception and format response
```

```
    @ExceptionHandler(ResourceNotFoundException.class)
    public ResponseEntity handleNotFound(ResourceNotFoundException ex) {
        // Returns HTTP 404 with our custom message in the body
        return new ResponseEntity<>(ex.getMessage(), HttpStatus.NOT_FOUND);
    }

    // Catch-all safety net for general crashes
    @ExceptionHandler(Exception.class)
    public ResponseEntity handleGeneralError(Exception ex) {
        return new ResponseEntity<>("Internal Server Error occurred.", HttpStatus.INTERN
    }
}
```

```
Expected Output / API Response:
API RESPONSE (HTTP 404 Not Found):
User ID 99 does not exist.
```

Code Explanation:
If any Service in the entire application executes `throw new ResourceNotFoundException("User ID 99 does not exist")`, the `@ControllerAdvice` grabs it instantly. It builds an HTTP 404 Response containing our clean message.

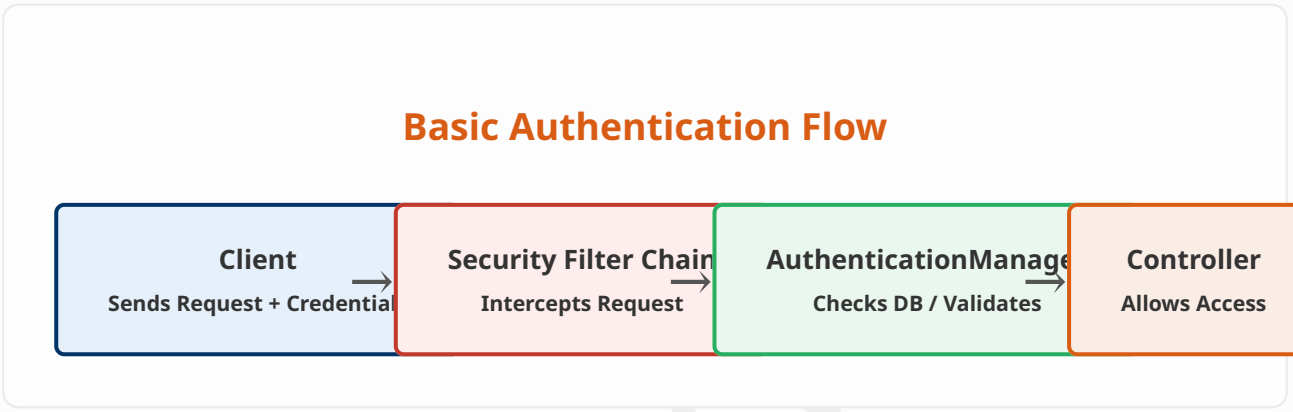
> ***Real-world Use:***
> *Standardizing API errors across a massive microservice, ensuring mobile app developers always receive a consistent `{ "error": "...", "code": 404 }` JSON format when things go wrong.*

> **Common Mistakes:**
>
> - Leaking raw Java stack traces to the client. This exposes internal system architecture and is a severe security vulnerability. Always use a global exception handler.

> **Best Practices:**
>
> - Create a custom `ErrorResponse` DTO class containing a timestamp, message, and status code, and return that from your Exception Handler for highly professional API design.

10. Authentication Basics & Spring Security

Spring Security is a highly customizable authentication and access-control framework. It is the de facto standard for securing Spring-based applications.



When you add the `spring-boot-starter-security` dependency to Maven, Spring Boot automatically secures all HTTP endpoints by default. If a client attempts a GET request without a valid username and password in the HTTP Authorization header, Spring Security instantly rejects it with an **HTTP 401 Unauthorized** response, completely shielding your Controllers.

IT STUDY

11. Project: Complete RESTful Backend Using Spring Boot

We will build a comprehensive Student Management System. We will utilize the 3-Tier Architecture, Spring Data JPA for database access, DTOs for safe data transfer, and a Global Exception Handler.

1. Project Structure & Setup

You generate a project using *start.spring.io* (Spring Initializr) with the following dependencies: **Spring Web**, **Spring Data JPA**, **MySQL Driver**, and **Validation**.

```
src/main/java/com/itstudyhub/student/
 ├── StudentApplication.java       (Main Boot Class)
 ├── controller/
 │    └── StudentController.java   (API Endpoints)
 ├── service/
 │    └── StudentService.java      (Business Logic)
 ├── repository/
 │    └── StudentRepository.java   (Database Interface)
 ├── model/
 │    └── Student.java             (JPA Entity)
 ├── dto/
 │    └── StudentDTO.java          (Data Transfer Object)
 └── exception/
      └── GlobalExceptionHandler.java
```

2. Configuration (application.properties)

```
# Connects Spring to MySQL. (Update with your local DB credentials)
spring.datasource.url=jdbc:mysql://localhost:3306/school_db
spring.datasource.username=root
spring.datasource.password=your_password
```

```
# Automatically creates the database table based on our @Entity class!
spring.jpa.hibernate.ddl-auto=update
```

3. Entity & Repository

```
import jakarta.persistence.*;
import org.springframework.data.jpa.repository.JpaRepository;

@Entity
public class Student {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String name;
    private String email;

    // Getters, Setters, and empty Constructor omitted for brevity
}

// Spring automatically generates the SQL for us!
public interface StudentRepository extends JpaRepository {
    boolean existsByEmail(String email); // Custom magic method
}
```

4. Data Transfer Object (DTO) & Exception

```
import jakarta.validation.constraints.*;

public class StudentDTO {
    @NotBlank(message = "Name is required")
    public String name;

    @Email(message = "Invalid email format")
    public String email;
}

// Custom Exception
class DuplicateEmailException extends RuntimeException {
```

```
    public DuplicateEmailException(String msg) { super(msg); }
}
```

5. Service Layer (Business Logic)

```
import org.springframework.stereotype.Service;
import java.util.List;

@Service
public class StudentService {
    private final StudentRepository repository;

    // Constructor Injection
    public StudentService(StudentRepository repository) {
        this.repository = repository;
    }

    public Student addStudent(StudentDTO dto) {
        if (repository.existsByEmail(dto.email)) {
            throw new DuplicateEmailException("Email already registered!");
        }
        Student student = new Student();
        student.setName(dto.name);
        student.setEmail(dto.email);
        return repository.save(student); // Saves to MySQL!
    }

    public List getAllStudents() {
        return repository.findAll();
    }
}
```

6. Controller Layer (REST API) & Exception Handler

```
import org.springframework.web.bind.annotation.*;
import jakarta.validation.Valid;
import java.util.List;
```

```
import org.springframework.http.ResponseEntity;
import org.springframework.http.HttpStatus;

@RestController
@RequestMapping("/api/students")
public class StudentController {

    private final StudentService service;

    public StudentController(StudentService service) {
        this.service = service;
    }

    @PostMapping
    public Student create(@Valid @RequestBody StudentDTO dto) {
        // Controller hands sanitized DTO to Service
        return service.addStudent(dto);
    }

    @GetMapping
    public List readAll() {
        return service.getAllStudents();
    }
}

@ControllerAdvice
class GlobalHandler {
    @ExceptionHandler(DuplicateEmailException.class)
    public ResponseEntity handleDuplicate(DuplicateEmailException ex) {
        return new ResponseEntity<>(ex.getMessage(), HttpStatus.CONFLICT); // 409 Confli
    }
}
```

```
API REQUEST (POST /api/students):
{ "name": "Alice", "email": "alice@test.com" }

API RESPONSE (HTTP 200 OK):
{ "id": 1, "name": "Alice", "email": "alice@test.com" }

---
API REQUEST 2 (POST /api/students):
```

```
{ "name": "Bob", "email": "alice@test.com" }

API RESPONSE 2 (HTTP 409 Conflict):
Email already registered!
```

Project Architecture Explanation:

1. **Control Flow:** The Client hits the `@PostMapping`. The Controller validates the JSON body using `@Valid`. If it passes, the `StudentDTO` is handed to the `StudentService`.
2. **Business Logic:** The Service checks the database using `repository.existsByEmail()`. If it exists, it manually throws a custom exception. If not, it converts the DTO into an Entity and calls `repository.save()`.
3. **Exception Interception:** When the `DuplicateEmailException` is thrown, the `@ControllerAdvice` intercepts it globally, returning a clean HTTP 409 response instead of crashing the server.

IT STUDY HUB

12. Module Assessment and Interview Prep

Test your mastery of Spring Boot, Dependency Injection, and REST architectures to prepare for enterprise backend interviews.

A. Practice Questions (Test Your Logic)

1. What annotation is used to mark a class as a Spring Boot entry point?
2. Write the Controller method signature to handle a `DELETE` request to `/api/users/{id}`.
3. Why is Constructor Injection preferred over Field Injection (`@Autowired` on a variable)?
4. Create a DTO class for a `Product` with validation annotations ensuring the `price` is positive and `name` is not blank.
5. Write a `JpaRepository` interface for a `Book` entity with a Long primary key.
6. Add a custom method to the repository in Question 5 that finds books by `author`.
7. Identify the error: A Controller accepts an `@Entity` object directly in its `@RequestBody` without a DTO.
8. What happens if you define a `@Service` but forget to put the annotation above the class?
9. Explain how a `@ControllerAdvice` class prevents leaking Java stack traces to the frontend.
10. Write the `application.properties` configuration to run your Tomcat server on port 8081.
11. What is the difference between `@RequestParam` and `@PathVariable`?
12. Map the standard HTTP Methods (GET, POST, PUT, DELETE) to their corresponding `@Mapping` annotations.
13. Write a custom exception `UserNotFoundException`.
14. Write the `@ExceptionHandler` method inside a GlobalExceptionHandler to catch the exception from Question 13.
15. Explain the concept of "Inversion of Control" using a real-world analogy.

IT STUDY HUB

B. University Viva Questions & Answers

Q: What is the Spring Framework?

Ans: An open-source, enterprise-level application framework for Java that provides infrastructure support, allowing developers to focus on business logic rather than boilerplate plumbing.

Q: What is the main difference between Spring and Spring Boot?

Ans: Spring requires extensive manual configuration (XML/Java) and external server deployment. Spring Boot relies on auto-configuration ('Convention over Configuration') and embeds a Tomcat server, allowing apps to run standalone immediately.

Q: What is Inversion of Control (IoC)?

Ans: A design principle where the control of object creation and lifecycle management is transferred from the programmer's manual code (using 'new') to a framework container (the Spring ApplicationContext).

Q: What is Dependency Injection (DI)?

Ans: The practical implementation of IoC. It is a design pattern where the Spring container automatically provides (injects) the required dependencies (objects) into a class at runtime, typically via the constructor.

Q: What is a Spring Bean?

Ans: Any Java object that is instantiated, assembled, and managed by the Spring IoC container.

Q: What does @SpringBootApplication do?

Ans: It is a convenience annotation that combines @Configuration, @EnableAutoConfiguration, and @ComponentScan, essentially booting up the application and telling Spring where to look for components.

Q: What is Component Scanning?

Ans: The process where Spring automatically scans your project's packages on startup to discover classes annotated with @Component, @Service, @Repository, or @Controller, registering them as Beans in the container.

Q: Explain the 3-Tier Architecture in Spring.

Ans: Controller Layer (handles HTTP web requests and JSON mapping), Service Layer (handles core business logic and math), Repository Layer (handles data access and SQL generation via JPA).

IT STUDY HUB

Q: What is a REST API?

Ans: Representational State Transfer. An architectural style that uses standard HTTP methods (GET, POST, PUT, DELETE) to interact with stateless resources over the web, typically returning JSON data.

Q: What is the difference between @Controller and @RestController?

Ans: @Controller is used in legacy web apps to return HTML view pages. @RestController combines @Controller and @ResponseBody, bypassing view resolution to return raw JSON data directly to the client.

Q: What is the difference between @PathVariable and @RequestParam?

Ans: @PathVariable extracts values from the URI path itself (e.g., /users/5). @RequestParam extracts values from the query string parameters appended after a question mark (e.g., /users?id=5).

Q: What does @RequestBody do?

Ans: It instructs Spring to deserialize the incoming HTTP request body (usually JSON text) directly into a mapped Java Object (DTO) for use in the controller method.

Q: What is an Entity in JPA?

Ans: A lightweight, persistent domain object. An @Entity class represents a table in a relational database, and each instance of the class represents a row in that table.

Q: What is Spring Data JPA?

Ans: An abstraction layer over standard JPA/Hibernate that dramatically reduces database boilerplate. It automatically generates SQL implementations for standard repository interfaces at runtime based on method names.

Q: What does spring.jpa.hibernate.ddl-auto=update do?

Ans: It commands Hibernate to inspect your @Entity classes during application startup and automatically alter or generate the corresponding database tables to match your Java code schema.

Q: Why should we use DTOs (Data Transfer Objects)?

Ans: To decouple the database Entity layer from the presentation layer. DTOs prevent sensitive database fields (like passwords) from being accidentally exposed to the client, and they prevent hackers from over-posting data into the database.

IT STUDY HUB

Q: What does the @Valid annotation do?

Ans: It triggers the Jakarta Bean Validation framework to inspect the incoming DTO object against defined constraints (@NotBlank, @Min, @Email) before the controller method executes. If it fails, Spring throws an exception.

Q: What is @ControllerAdvice?

Ans: A global interceptor that allows developers to write consolidated, application-wide Exception Handlers. It catches exceptions thrown by any controller and formats a unified HTTP error response.

Q: What is Spring Security?

Ans: A highly customizable framework focused on providing Authentication (verifying who you are) and Authorization (verifying what you are allowed to do) for Java applications.

Q: What is Constructor Injection and why is it preferred?

Ans: It is the process of injecting dependencies via a class constructor rather than annotating the fields directly. It is preferred because it forces dependencies to be provided at instantiation, making the class state safer, immutable (using 'final'), and easier to unit test without starting the Spring container.

C. Intermediate Interview Questions

Interview Q: How exactly does Spring Boot Auto-Configuration work under the hood?

Ans: Spring Boot uses the @EnableAutoConfiguration annotation to scan your project's classpath (the jars in your pom.xml). Based on the libraries it finds, it registers default Beans. For example, if it sees 'spring-boot-starter-data-jpa' and a MySQL driver, it automatically creates the DataSource and EntityManager beans without you writing any connection code.

Interview Q: Explain the difference between a Singleton Bean and a Prototype Bean.

Ans: By default, all Spring Beans are Singletons, meaning the IoC container creates exactly ONE instance of the object and shares that exact same instance every time it is injected anywhere in the app. A Prototype scope tells Spring to create a brand new instance of the object every single time it is requested.

IT STUDY HUB

Interview Q: Why is Field Injection (@Autowired on private variables) considered bad practice?

Ans: It violates the OOP principle of Encapsulation, making the class tightly coupled to the Spring framework. Without Spring, the class cannot be easily instantiated or tested because there is no constructor to pass the mock dependencies into. It also prevents the use of 'final' fields, making the class state mutable and prone to NullPointerExceptions.

Interview Q: How does Hibernate map a Java object to a database row?

Ans: Hibernate uses Reflection to inspect the @Entity class. It reads annotations like @Table and @Column to map Java properties to SQL schema definitions. When saving, it uses an internal Session to track object state changes and translates those changes into optimized SQL INSERT/UPDATE statements, abstracting the JDBC layer entirely.

Interview Q: Explain the N+1 SELECT problem in Hibernate.

Ans: The N+1 problem occurs when fetching a collection of entities that have a One-to-Many relationship (e.g., loading 10 Authors, and then looping through them to print their Books). Hibernate will execute 1 query to fetch the Authors, and then N separate queries to fetch the Books for each author, destroying performance. It is solved by using 'JOIN FETCH' or EntityGraphs to load all data in a single SQL query.

Interview Q: What is the difference between Authentication and Authorization?

Ans: Authentication verifies Identity (Are you who you say you are? e.g., verifying an email and password). Authorization verifies Access (Are you allowed to do this action? e.g., verifying if the user has the 'ADMIN' role required to delete a record).

Interview Q: How would you secure a stateless REST API using Spring Security?

Ans: Since REST APIs are stateless, we cannot use server-side Sessions. We use JWT (JSON Web Tokens). Upon login, the server generates a cryptographically signed JWT containing user roles and returns it. The client stores it and attaches it to the 'Authorization: Bearer' header of every subsequent HTTP request. Spring Security intercepts the request, validates the token signature, and grants access.

Interview Q: What is the difference between PUT and PATCH HTTP methods?

Ans: PUT is strictly for replacing an entire resource. If a User object has 5 fields and you send a PUT with only 1 field, the other 4 fields should theoretically be wiped to null. PATCH is for partial updates; it applies partial modifications only to the specific fields provided in the JSON body, leaving the rest of the object intact.

IT STUDY HUB

Interview Q: What is the purpose of ResponseEntity in a REST Controller?

Ans: While returning a raw object (like StudentDTO) relies on Spring to assume an HTTP 200 OK status, returning a ResponseEntity gives the developer full control over the HTTP response. It allows you to explicitly set the HTTP Status Code (e.g., 201 Created, 404 Not Found) and inject custom HTTP Headers into the response.

Interview Q: Describe the process of setting up Global Exception Handling.

Ans: Create a class and annotate it with @ControllerAdvice. Create methods returning ResponseEntity and annotate them with @ExceptionHandler(YourException.class). Inside the method, extract the message from the thrown exception, package it into the ErrorDTO, set the appropriate HTTP status code (e.g., HttpStatus.BAD_REQUEST), and return it. This cleanly decouples all error-handling logic from your controllers.

IT STUDY HUB

13. Module Summary and Key Terms

Key Terms Glossary

- **Dependency Injection (DI):** Passing required objects into a class (usually via constructor) rather than creating them manually.
- **IoC Container:** The core of Spring that instantiates, manages, and injects Beans automatically.
- **Bean:** A Java object completely managed by the Spring IoC container.
- **DTO:** Data Transfer Object. A stripped-down object used to securely pass data between the Client and Controller.
- **ORM:** Object-Relational Mapping (e.g., Hibernate). A tool that automatically translates Java objects into SQL tables.
- **Auto-Configuration:** Spring Boot's magical ability to configure database and server settings automatically based on classpath dependencies.

Module Summary

In this module, you ascended to the level of **Enterprise Backend Developer**. You abandoned manual JDBC configuration and embraced the immense power of **Spring Boot**. You learned how to relinquish control of object creation to the **IoC Container**, utilizing **Dependency Injection** to cleanly decouple your components.

You implemented the strict **3-Tier Architecture**. You exposed your application to the web using `@RestController`, parsing incoming JSON with `@RequestBody`. You processed complex business logic in the `@Service` layer. You connected to databases without writing a single line of SQL using **Spring Data JPA** and **Hibernate**. Finally,

you secured your application borders by validating **DTOs** and catching application-wide errors gracefully with a Global Exception Handler.

WHAT COMES NEXT?

MODULE 12 — ADVANCED JAVA, TESTING & PRODUCTION

You have built a fully functional enterprise backend. But how do you guarantee it won't break when another developer modifies your code? And how do you put it on the internet for the world to use? In the final volume of this series, we cover **Unit Testing** with JUnit and Mockito, advanced CI/CD pipelines, Docker containerization, and deploying your Spring Boot application to a live production server.

IT STUDY HUB SERIES